

NBA DASHBOARD

SP26_CS-GY_6083 | Aspen | Paola Boyd

8 Tables | 2 Triggers | 3 Functions | 3 Views

Full CRUD | 3 Reports | 3NF Normalized | Web Scraped from ESPN

<https://github.com/paoboyd/NBA-Dashboard>

Project Summary

A full-stack NBA dashboard that scrapes real player data from ESPN, stores it in PostgreSQL with 8 normalized tables, and serves it through a Flask web app. Users pick one team at a time to view roster, salary breakdown, and 5-year player stats.

scraper.py

Web scraper using requests + ESPN JSON API

database.py

PostgreSQL schema: 8 tables, 3 functions, 2 triggers, 3 views

main.py

Flask web server with REST API endpoints

seed_data.py

Fallback data for all 30 NBA teams / 150+ players

templates/index.html

Interactive dashboard UI with team selector

8 Tables

Properly designed with foreign keys

★ = Primary Key → = Foreign Key

teams

★ team_id
name
city
abbreviation
conference
division
arena

players

★ player_id
→ team_id
first_name
last_name
position
salary
jersey_number

player_stats

★ stat_id
→ player_id
season_year
ppg, rpg, apg
fg_pct, three_pct
games_played

positions

★ position_id
position_name
description

contracts

★ contract_id
→ player_id
→ team_id
salary
start_date
end_date
contract_type

injuries

★ injury_id
→ player_id
injury_type
date_injured
date_returned
games_missed

rosters

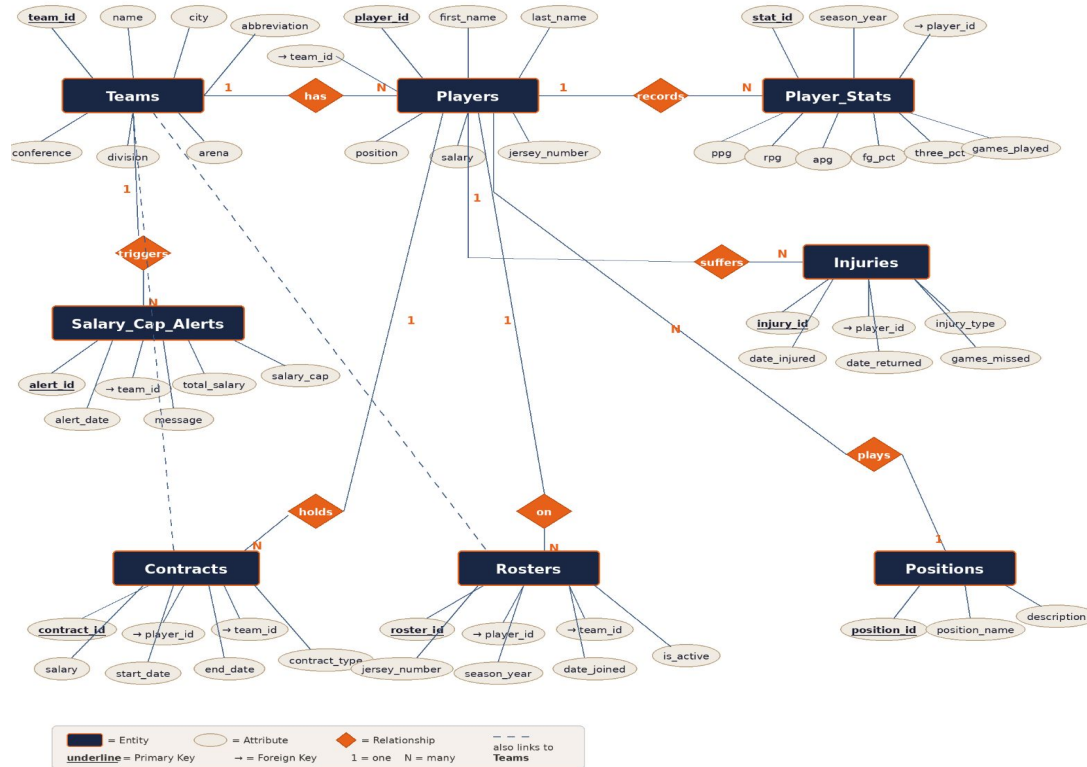
★ roster_id
→ player_id
→ team_id
jersey_number
season_year
date_joined
is_active

salary_cap_alerts

★ alert_id
→ team_id
total_salary
salary_cap
alert_date
message

Code: database.py → init_db()

ER Diagram



Triggers

2 triggers with business logic

Trigger 1: Roster deactivation

```
CREATE OR REPLACE FUNCTION
  deactivate_old_roster_entries()
RETURNS TRIGGER AS $$
BEGIN
  UPDATE rosters
  SET is_active = FALSE
  WHERE player_id = NEW.player_id
    AND roster_id != NEW.roster_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_deactivate_old_roster
AFTER INSERT ON rosters
FOR EACH ROW
EXECUTE FUNCTION
  deactivate_old_roster_entries();
```

Trigger 2: Salary cap alert

```
CREATE OR REPLACE FUNCTION
  log_salary_cap_alert()
RETURNS TRIGGER AS $$
DECLARE
  v_team_total DOUBLE PRECISION;
BEGIN
  SELECT get_team_total_salary(
    NEW.team_id) INTO v_team_total;
  IF NEW.salary > 140000000 THEN
    INSERT INTO salary_cap_alerts
      (team_id, total_salary,
       salary_cap, alert_date, message)
    VALUES (NEW.team_id, v_team_total,
      140000000, NOW()::TEXT,
      'Contract exceeds cap');
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

Database Tables

8 tables with complexity and constraints

```
CREATE TABLE IF NOT EXISTS teams (  
    team_id    SERIAL PRIMARY KEY,  
    name       TEXT NOT NULL UNIQUE,  
    city       TEXT NOT NULL,  
    abbreviation TEXT NOT NULL,  
    conference TEXT NOT NULL CHECK (conference IN ('Eastern','Western')),  
    division   TEXT NOT NULL,  
    arena      TEXT NOT NULL,  
    founded_year INTEGER NOT NULL,  
    primary_color TEXT DEFAULT '#333333',  
    accent_color TEXT DEFAULT '#CCCCC'  
);
```

```
CREATE TABLE IF NOT EXISTS players (  
    player_id    SERIAL PRIMARY KEY,  
    first_name   TEXT NOT NULL,  
    last_name    TEXT NOT NULL,  
    team_id      INTEGER REFERENCES teams(team_id),  
    position     TEXT,  
    salary       DOUBLE PRECISION DEFAULT 0,
```

```
    jersey_number INTEGER,
```

```
    contract_type TEXT DEFAULT 'Standard'
```

```
); Code: database.py → init_db(), lines ~20-125
```

```
-- + 6 more tables: positions, player_stats, injuries,
```

```
    contracts, rosters, salary_cap, alerts
```

Functions

3 stored functions with complexity

get_team_total_salary(team_id)

Returns sum of all player salaries for a given team

```
SELECT COALESCE(SUM(salary), 0)
FROM players
WHERE team_id = p_team_id;
```

get_player_tenure(player_id)

Calculates years since player joined via roster table

```
SELECT COALESCE(
    EXTRACT(YEAR FROM AGE(
        NOW(), MIN(date_joined::DATE))),
    0)
FROM rosters WHERE player_id = p_id;
```

get_team_avg_age(team_id)

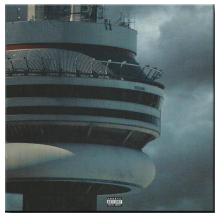
Returns average age of all players on a team from DOB

```
SELECT COALESCE(
    AVG(EXTRACT(YEAR FROM
        AGE(NOW(), dob::DATE))), 0)
FROM players
WHERE team_id = p_team_id;
```

Code: database.py → init_db(), lines ~130-165

Views

3 views with complexity



```
-- VIEW 1: Main dashboard (teams + players + stats)
CREATE OR REPLACE VIEW vw_team_dashboard AS
SELECT t.team_id, t.name AS team_name, p.first_name, p.last_name,
       p.position, p.salary, ps.ppg, ps.rpg, ps.apg, ps.fg_pct
FROM teams t
LEFT JOIN players p ON t.team_id = p.team_id
LEFT JOIN player_stats ps ON p.player_id = ps.player_id
   AND ps.season_year = (SELECT MAX(ps2.season_year)
                        FROM player_stats ps2 WHERE ps2.player_id = p.player_id);

-- VIEW 2: Salary breakdown by position per team
CREATE OR REPLACE VIEW vw_salary_breakdown AS
SELECT t.name, p.position, COUNT(*), AVG(salary), SUM(salary), MAX(salary)
FROM teams t JOIN players p ON t.team_id = p.team_id
GROUP BY t.team_id, t.name, p.position;

-- VIEW 3: Injury report with player + team info
CREATE OR REPLACE VIEW vw_injury_report AS
SELECT i.*, p.first_name, p.last_name, t.name AS team_name
FROM injuries i JOIN players p ON i.player_id = p.player_id
JOIN teams t ON p.team_id = t.team_id
ORDER BY i.date_injured DESC;
Code: database.py → init_db(), lines ~225-274
```


Select Statements

All tables have **SELECT** statements

```
-- Teams: Western Conference
SELECT * FROM teams WHERE conference = 'Western' ORDER BY city;

-- Players: Full roster with stats for a team (multi-table JOIN)
SELECT p.first_name || ' ' || p.last_name AS name, p.position, p.salary,
       ps.ppg, ps.rpg, ps.apg, ps.fg_pct
FROM players p
LEFT JOIN player_stats ps ON p.player_id = ps.player_id
      AND ps.season_year = (SELECT MAX(season_year) FROM player_stats ps2
                           WHERE ps2.player_id = p.player_id)
WHERE p.team_id = 14 ORDER BY p.salary DESC;

-- Contracts: All max contracts sorted by salary
SELECT p.first_name || ' ' || p.last_name AS player, t.name AS team,
       c.salary, c.contract_type
FROM contracts c JOIN players p ON c.player_id = p.player_id
JOIN teams t ON c.team_id = t.team_id
WHERE c.contract_type = 'Max' ORDER BY c.salary DESC;

-- All 8 tables have SELECT statements
```

Code: `database.py` → `get_all_teams()`, `get_team_roster()`, `get_league_leaders()` + `nba_dashboard_postgresql.sql`

Insert Statements

All tables have INSERT statements

```
-- Teams
INSERT INTO teams (name, city, abbreviation, conference, division, arena,
    founded_year) VALUES ('Sonics', 'Seattle', 'SEA', 'Western', 'Northwest',
    'Climate Pledge Arena', 1967);

-- Players (web-scraped from ESPN via scraper.py)
INSERT INTO players (first_name, last_name, jersey_number, team_id,
    position, salary, country)
VALUES ('LeBron', 'James', 23, 14, 'PF', 47607350, 'USA');

-- Player Stats
INSERT INTO player_stats (player_id, season_year, games_played,
    ppg, rpg, apg, fg_pct)
VALUES (1, '2025-26', 55, 25.7, 7.3, 8.3, 0.540);

-- Contracts (triggers salary cap alert automatically)
INSERT INTO contracts (player_id, team_id, salary, start_date,
    end_date, contract_type)
VALUES (1, 14, 47607350, '2024-07-01', '2027-06-30', 'Max');
```

```
-- All 8 tables have INSERT statements
```

Code: *scraper.py* → *scrape_with_nba_api()* + *seed_data.py* → *seed_all()* + *nba_dashboard_postgresql.sql*

Update Statements

All tables have UPDATE statements

```
-- Teams: Update arena name
UPDATE teams SET arena = 'Intuit Dome' WHERE abbreviation = 'LAC';

-- Players: Salary extension
UPDATE players SET salary = 52000000, contract_type = 'Max'
WHERE first_name = 'Jayson' AND last_name = 'Tatum';

-- Player Stats: Mid-season update
UPDATE player_stats SET games_played = 65, ppg = 26.0
WHERE player_id = 1 AND season_year = '2025-26';

-- Injuries: Player returned from injury
UPDATE injuries SET date_returned = '2026-04-20', games_missed = 10
WHERE player_id = 11 AND date_returned IS NULL;

-- Contracts: Extension
UPDATE contracts SET salary = 58000000, end_date = '2030-06-30'
WHERE player_id = 6 AND team_id = 2;
```

```
-- All 8 tables have UPDATE statements
```

Code: *nba_dashboard_postgresql.sql* – Section 8

Delete Statements

All tables have well-structured DELETE statements

```
-- Teams: Remove a team
DELETE FROM teams WHERE abbreviation = 'SEA';

-- Players: Waive a player
DELETE FROM players WHERE first_name = 'Bronny' AND last_name = 'James';

-- Player Stats: Remove erroneous entry
DELETE FROM player_stats WHERE player_id = 11 AND season_year = '2020-21';

-- Injuries: Remove record logged in error
DELETE FROM injuries WHERE player_id = 11 AND injury_type = 'Knee Sprain';

-- Contracts: Void a future contract
DELETE FROM contracts WHERE player_id = 3 AND start_date > CURRENT_DATE;

-- Rosters: Remove player from roster (waived)
DELETE FROM rosters WHERE player_id = 22 AND season_year = '2026-27';

-- Salary Cap Alerts: Clear old alerts
DELETE FROM salary_cap_alerts
WHERE alert_date < CURRENT_TIMESTAMP - INTERVAL '30 days';
```

Code: nba_dashboard_postgresql.sql – Section 8

Normalization — 3NF

Fully addressed in presentation

1NF

First Normal Form

All columns are atomic — no arrays, no repeating groups. Each cell holds exactly one value. Every row is uniquely identified by a primary key (SERIAL auto-increment).

position = 'PG' (one value, not 'PG, SG')

2NF

Second Normal Form

No partial dependencies. Every non-key column depends on the ENTIRE primary key. Composite unique constraints enforce this (player_id + season_year in player_stats).

ppg depends on BOTH player AND season, not just one

3NF

Third Normal Form

No transitive dependencies. Team info (arena, conference) lives in teams table only — not repeated in players. We store just team_id as a foreign key.

arena is NOT in players table — only team_id FK

Demonstrated across all tables in database.py

Isolation Levels

What isolation level, where, and why

READ COMMITTED

PostgreSQL default

Where: All standard dashboard SELECT queries

Why: Prevents dirty reads while allowing good concurrency for read-heavy dashboard usage

Example: `SELECT * FROM vw_team_dashboard WHERE team_name = 'Lakers';`

SERIALIZABLE

Strictest level

Where: Salary cap trigger function

Why: Prevents race conditions where two concurrent contract inserts could both pass the cap check but together exceed it

Example: `BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE; INSERT INTO contracts ...; COMMIT;`

Applied in database.py triggers + dashboard queries

Integrity Enforcement

All integrity types with examples

Entity

PRIMARY KEY (SERIAL) on every table

```
player_id SERIAL PRIMARY KEY
```

Referential

FOREIGN KEY linking tables together

```
team_id INTEGER REFERENCES teams(team_id)
```

Domain

CHECK constraints limiting valid values

```
CHECK (conference IN ('Eastern','Western'))  
CHECK (salary > 0)
```

Key

UNIQUE constraints preventing duplicates

```
UNIQUE(player_id, season_year)  
name TEXT NOT NULL UNIQUE
```

NOT NULL

Required fields cannot be empty

```
first_name TEXT NOT NULL  
injury_type TEXT NOT NULL
```

Code: `database.py` → `CREATE TABLE` statements

Reports with Aggregation

3 complex reports with GROUP BY, HAVING, aggregate functions

```
-- REPORT 1: Team salary by position
SELECT t.name, p.position, COUNT(*), AVG(p.salary),
       SUM(p.salary), get_team_total_salary(t.team_id)
FROM teams t JOIN players p ON t.team_id = p.team_id
GROUP BY t.name, t.team_id, p.position ORDER BY SUM(p.salary) DESC;

-- REPORT 2: Performance vs. salary (value index)
SELECT p.first_name||' '||p.last_name, ps.ppg, p.salary,
       ROUND((ps.ppg + ps.rpg + ps.apg) /
             NULLIF(p.salary / 1000000.0, 0), 2) AS value_index
FROM players p JOIN player_stats ps ON p.player_id = ps.player_id
WHERE ps.ppg > 10 ORDER BY value_index DESC;

-- REPORT 3: Injury impact by team
SELECT t.name, COUNT(i.injury_id) AS total_injuries,
       SUM(i.games_missed) AS total_games_missed,
       AVG(i.games_missed) AS avg_missed,
       STRING_AGG(DISTINCT i.injury_type, ', ') AS types
FROM teams t JOIN players p ON t.team_id = p.team_id
LEFT JOIN injuries i ON p.player_id = i.player_id
GROUP BY t.team_id, t.name HAVING COUNT(i.injury_id) > 0;
```

Code: database.py → get_team_salary_breakdown() + nba_dashboard_postgresql.sql Section 9

Web Scraping — ESPN

538 players scraped across all 30 teams

```
# scraper.py — Pulls real data from ESPN's JSON APIs
import requests

HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ...'
}

# ESPN API endpoints
ESPN_TEAMS = 'site.api.espn.com/.../nba/teams'
ESPN_ROSTER = 'site.api.espn.com/.../teams/{id}/roster'
ESPN_ATHLETE = 'site.web.api.espn.com/.../athletes/{id}'

def scrape_with_nba_api():
    # fetch all 30 team IDs from ESPN
    data = requests.get(ESPN_TEAMS, headers=HEADERS).json()

    # for each team, fetch full roster (15-22 players)
    for team in teams:
        roster = requests.get(ROSTER_URL.format(id=team_id)).json()
        for athlete in roster['athletes']:
            # parse: name, height, weight, jersey, position, salary
            # insert into PostgreSQL players table

Code: scraper.py
# fetch per-player stats (PPG, RPG, APG, FG%)
for athlete_id in all_players:
```

LIVE DEMO

replit.com — NBA Dashboard

Pick a team → View roster, salary, stats

Click a player → 5-year stat history

Search bar → Find any player across the league